HWRS 640 – Computational Methods for Data Driven Earth Science

Name: Maria Castro
Student ID: 23953239
Due Date: April 24 2026, 11:59 PM
Assignment 4
Github Repository: `https://github.com/mariaandradeal/HWRS640_HW4/`

Problem 1: Data access and exploratory analysis Use 'minicamels' to inspect the dataset and build your supervised learning problem. Your 'data.py' module should contain the core code for dataset access, sample generation, and PyTorch 'Dataset'/'DataLoader' creation.

1. Load the basin index, static attributes, and at least several basin time series using MiniCamels.

2. Report the dataset structure: number of basins, time span, dynamic input variables, target variable, and number of static attributes used.

   The dataset consists of 50 basins spanning the period from October 1, 1980, to September 30, 2010. For each basin, daily meteorological forcings are provided, including precipitation (`prcp`), maximum temperature (`tmax`), minimum temperature (`tmin`), shortwave radiation (`srad`), and vapor pressure (`vp`). The prediction target is the observed streamflow (`qobs`).

   In addition to dynamic variables, 16 static basin attributes are available, including geographic location (latitude and longitude), topographic properties (elevation and slope), climatic indicators (mean precipitation, aridity), hydrologic indices (runoff ratio, baseflow index), soil characteristics, and vegetation properties. These attributes provide basin-specific context that can improve model performance when combined with temporal inputs.

3. Choose a train/validation/test split strategy and justify it in 3-4 sentences. You may split by basin, by time, or by a combination of both. Your split must avoid leakage.

   A time-based split was adopted independently for each basin in order to avoid temporal leakage. For every basin, the record was divided into three chronological subsets: training (through 2000-09-30), validation (2000-10-01 to 2005-09-30), and test (2005-10-01 to 2010-09-30). This approach preserves the temporal ordering of the hydrologic record and provides a realistic forecasting setting, where the model is trained on earlier years and evaluated on later unseen periods. To further prevent leakage, normalization statistics for both dynamic forcings and static attributes were computed using only the training portion of the dataset and then applied to the validation and test sets.

4. Define the supervised learning samples. Choose a sequence length (for example, 30, 60, or 90 days). Explain what the model input and target are. State whether you predict one day ahead or another forecast horizon.

   Supervised learning samples were constructed using a sliding window approach applied to each basin time series. A sequence length of 120 days was selected, such that each input sample consists of a continuous window of 120 daily observations of meteorological forcings. In addition to the dynamic inputs, each sample includes a vector of 16 static basin attributes representing physiographic and climatic characteristics. These static features are constant for a given basin and are incorporated alongside the temporal sequence to provide contextual information about hydrologic behavior.

   The prediction target is the observed streamflow (`qobs`) on the day immediately following the input sequence. Therefore, the model performs a one-step-ahead prediction, where the input window spans days $t - (seq_{len} - 1)$ to t, and the target corresponds to day $t + 1$. In the configuration used in this study, with $seq_{len} = 120$, the input window spans the previous 120 days. To improve numerical stability and reduce skewness in the streamflow distribution, the target variable was transformed using a $\log(1 + q)$ transformation during training and subsequently converted back to the original scale for evaluation.

5. Produce at least three exploratory plots, such as: streamflow time series for multiple basins, precipitation and streamflow for one basin over the same period, histograms of qobs, a map or scatterplot using static attributes

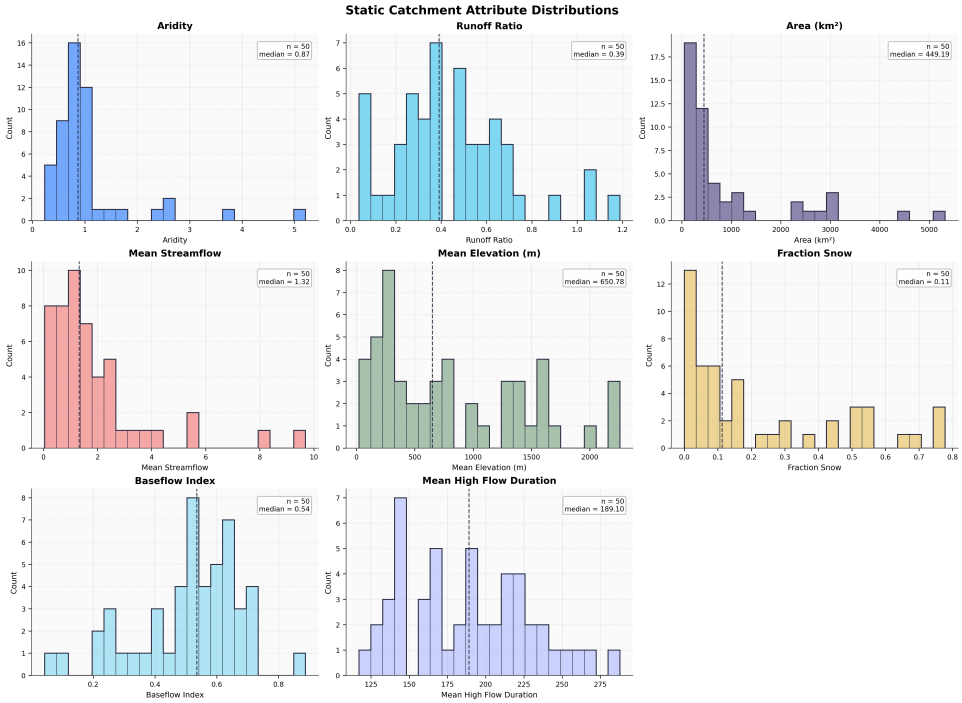

Figure 1: Histograms of static catchment attributes for 50 basins, including aridity, runoff ratio, drainage area, mean streamflow, mean elevation, fraction of snowfall, baseflow index, and mean high-flow duration. Dashed lines indicate median values for each variable.

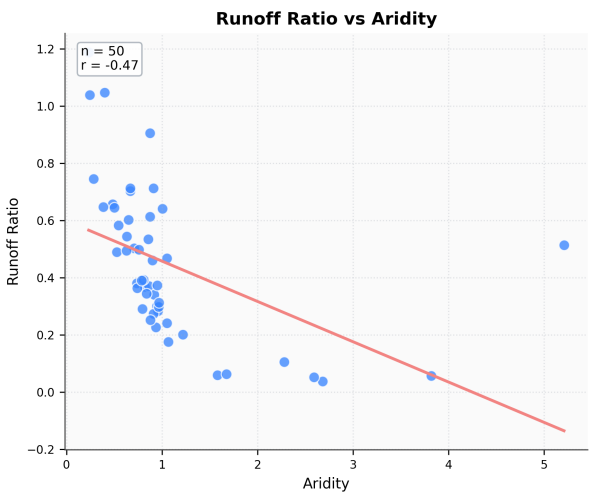

Figure 2: Scatter plot of runoff ratio versus aridity for 50 basins, showing a negative relationship ($r = -0.47$). The red line indicates the linear regression fit.

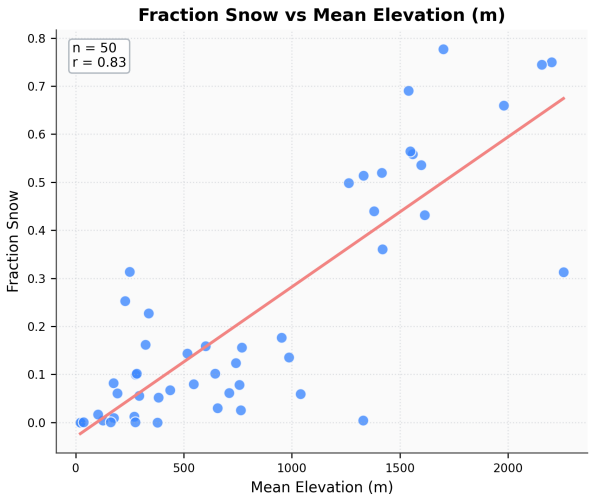

Figure 3: Scatter plot of fraction of snowfall versus mean basin elevation for 50 basins, showing a strong positive correlation ($r = 0.83$). The red line indicates the linear regression fit.

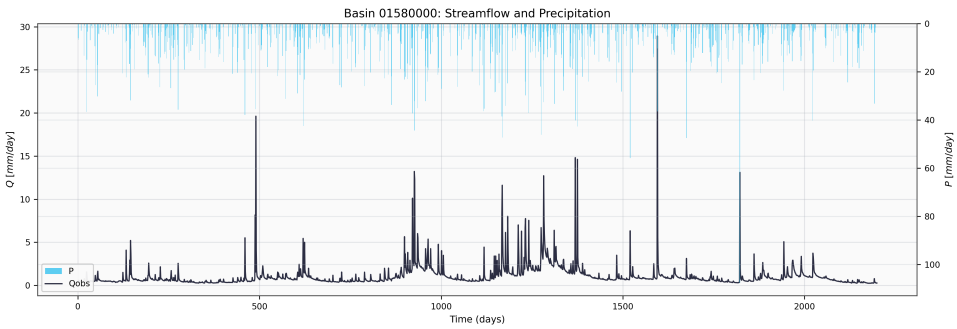

Figure 4: Time series of precipitation (P) and observed streamflow (Q_{obs}) for Basin 01580000.

6. Describe any preprocessing steps: missing-value handling, normalization or standardization, treatment of static attributes, sequence window construction.

   Missing values were handled by removing incomplete records prior to sequence construction. After the time series were cleaned, fixed-length input windows were generated, and an additional check was applied to discard any sequence containing missing values in the dynamic inputs or target. No imputation was used. This procedure ensures complete numerical inputs, although removing rows prior to sequence construction may reduce strict calendar continuity when gaps are present in the original time series.

   Dynamic input variables (e.g., precipitation, temperature, radiation, and vapor pressure) and the target variable were standardized using z-score normalization based on statistics computed from the training data. This transforms each variable

to have approximately zero mean and unit variance, improving numerical stability and convergence during training. In addition, the target variable can optionally be log-transformed to reduce skewness in streamflow distributions.

Static basin attributes (e.g., elevation, aridity, runoff ratio) were treated as auxiliary inputs and normalized using the same standardization approach as dynamic variables. These attributes were then concatenated with the dynamic sequence features at the model input stage, allowing the model to learn basin-specific behavior.

Supervised learning samples were constructed using a sliding window approach. For each basin time series, input sequences of fixed length (e.g., 60 days) were extracted from the dynamic variables. The corresponding target is the streamflow value (`qobs`) at the next time step (one-day-ahead prediction). This results in a dataset of overlapping sequences, where each sample consists of a sequence of meteorological inputs and a scalar streamflow target.

Problem 2: Model design Implement at least one sequence model in 'model.py'. Your model must:

1. Accept a batch of input sequences of shape (batch, $seq_{len}, num_{features}$) or an equivalent documented format.

2. Produce a streamflow prediction for each sample.

If you include static basin attributes, clearly explain how they enter the model. For example, you might concatenate static attributes to each timestep, encode them separately and fuse them with the sequence representation, or use another reasonable design. Document and discuss:

1. The full model architecture
   The implemented model is a sequence-to-one LSTM network with an explicit static-attribute fusion branch. For each sample, the model receives two inputs: (i) a dynamic meteorological sequence $\mathbf{x}_{seq}$ of shape $(batch, seq_{len}, num_{dynamic_features})$, and (ii) a static basin-attribute vector $\mathbf{x}_{static}$ of shape $(batch, num_{static_features})$. The output is a single scalar streamflow prediction for each sample, with shape $(batch, 1)$.

   The temporal branch is a unidirectional LSTM encoder. In the default configuration, the LSTM uses an input size equal to the number of dynamic variables, a hidden size of 64, and one recurrent layer. Because the layer is defined with `batch_first=True`, the input tensor is organized as $(batch, seq_{len}, num_{dynamic_features})$. After processing the full sequence, the LSTM returns the hidden-state tensor $\mathbf{h}_n$ of shape $(num_{layers}, batch, hidden_size)$. The model extracts the final hidden state from the last layer, $\mathbf{h}_n[-1]$, which yields a temporal representation of shape $(batch, 64)$. This vector summarizes the antecedent meteorological conditions over the input window.

The static branch is implemented as a small multilayer perceptron. The static input contains 16 basin attributes, which are passed through a linear layer mapping $\mathbb{R}^{16}$ to $\mathbb{R}^{32}$, followed by a ReLU activation and a dropout layer with probability 0.1. The output of this branch is therefore a static embedding of shape $(batch, 32)$. This branch provides a compact representation of basin-specific physiographic and climatic controls.

The temporal and static representations are then concatenated along the feature dimension, producing a fused vector of shape $(batch, 96)$, since $64 + 32 = 96$. This fused vector is passed through the regression head, which consists of a linear layer from 96 to 32 units, a ReLU activation, a dropout layer with probability 0.1, and a final linear layer from 32 to 1 unit. The final output is a single predicted streamflow value for each sample. Thus, the model combines a temporal encoder for recent forcing history, a static encoder for basin descriptors, and a fusion-based regression head for one-step-ahead prediction.

2. Why this architecture is appropriate for streamflow prediction

   This architecture is well suited for streamflow prediction because rainfall–runoff processes depend on both recent meteorological history and persistent basin characteristics. The LSTM is an appropriate temporal encoder because streamflow at a given day is influenced not only by the current forcing, but also by antecedent conditions such as previous rainfall, temperature, and atmospheric demand. By reading the forcing sequence one time step at a time and updating its hidden state, the LSTM can learn delayed and cumulative hydrologic responses, which are essential in catchment systems. The inclusion of static basin attributes might improve the physical relevance of the model. Basins with different elevation, aridity, soil depth, vegetation, slope, or runoff ratio may respond very differently to the same meteorological forcing.

3. One expected strength and one expected weakness of the architecture

   One expected strength of this architecture is its ability to jointly represent temporal dynamics and spatial heterogeneity. The LSTM branch can capture lagged and nonlinear effects of antecedent meteorological forcings, while the static branch provides persistent basin information that helps distinguish hydrologic behavior across catchments. One expected weakness is that the model compresses the entire meteorological sequence into a single final hidden-state vector before prediction. Although this summary is often effective, it may not fully preserve all relevant long-range information, especially for complex hydrologic regimes with multiple response timescales, strong seasonality, or snow-dominated behavior.

Problem 3: Training pipeline and CLI implementation Implement the training workflow in ‘train.py‘ and expose it through ‘main.py‘. Your training pipeline might include:

training and validation loops, loss computation, optimizer setup, model checkpointing, metric tracking, command-line arguments for key hyperparameters.

At minimum, your CLI training command should allow the user to specify: (i) model hyper parameters, (ii) sequence length, (iii) learning rate, (iv) batch size, (v) number of epochs, and (vi) output/checkpoint path.

Use a regression loss appropriate for streamflow prediction, such as MSE or MAE. You may additionally report Nash-Sutcliffe Efficiency (NSE), Kling-Gupta Efficiency (KGE), or correlation-based metrics during validation.

Produce the following plots:

1. training loss vs. epoch
2. validation loss vs. epoch
3. at least one validation metric vs. epoch

Your code should be organized so that plotting functions live in ‘visualization.py‘ and reusable helper functions live in ‘utils.py‘.

Fig. 5 shows that both training and validation losses decrease rapidly during the initial epochs, indicating effective learning of the underlying relationships. After approximately 6–8 epochs, the validation loss stabilizes while the training loss continues to decrease, suggesting mild overfitting. Overall, the relatively small gap between training and validation curves indicates good generalization performance, with the model achieving stable convergence within the training period.

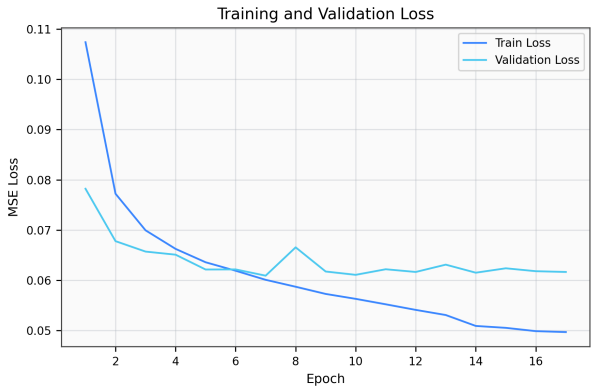

Figure 5: Training and validation loss as a function of epoch.

Fig. 6 shows that both curves show a rapid improvement during the initial epochs, indicating effective learning of streamflow dynamics. The validation NSE stabilizes after approximately 8–10 epochs, while the training NSE continues to increase slightly, suggesting mild overfitting. The relatively close agreement between training and validation NSE throughout training indicates good generalization performance and stable model convergence.

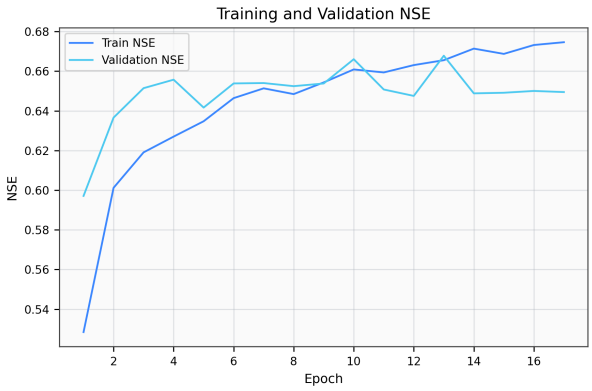

Figure 6: Evolution of the Nash–Sutcliffe Efficiency (NSE) for the training and validation sets over successive training epochs.

Problem 4: Evaluation and interpretation Evaluate your trained model on the held-out test set.

1. Report at least three quantitative metrics. At least one must be a hydrology-relevant metric such as NSE/KGE or peak flow/low flow.

   Model performance was evaluated using multiple quantitative metrics computed at the basin level, including the Nash–Sutcliffe Efficiency (NSE), Kling–Gupta Efficiency (KGE), Root Mean Square Error (RMSE), and Mean Absolute Error (MAE). All computed metrics are exported to a comma-separated values (CSV) file (`per_basin_metrics.csv`), which contains performance statistics for each basin, enabling reproducibility, detailed inspection, and further statistical analysis. In addition to tabular outputs, a consolidated boxplot visualization is generated to summarize the distribution of NSE, RMSE, MAE, and KGE across all basins.

   The results show generally strong performance for NSE and KGE across most basins, while RMSE and MAE exhibit greater variability, reflecting differences in error magnitude among catchments as presented in Fig. 7. This aggregated visualization provides a concise assessment of overall model skill and robustness across diverse hydrological conditions.

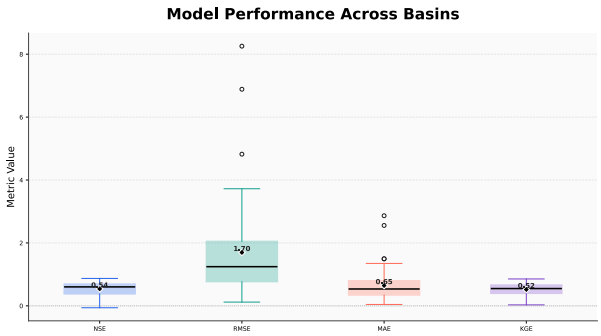

Figure 7: Distribution of basin-level model performance metrics across the MiniCAMELS dataset, including NSE, RMSE, MAE, and KGE. Boxplots summarize the median, interquartile range, and outliers for each metric, with mean values indicated within each box.

The curve in Fig. 8a shows that a large fraction of basins achieve moderate to high KGE values, indicating generally robust model skill with variability across hydrological conditions. The validation KGE in Fig. 8b increases rapidly during early epochs and stabilizes thereafter, indicating effective learning and limited overfitting. Together, these panels provide complementary perspectives on model performance: spatial variability across basins and temporal convergence during training.

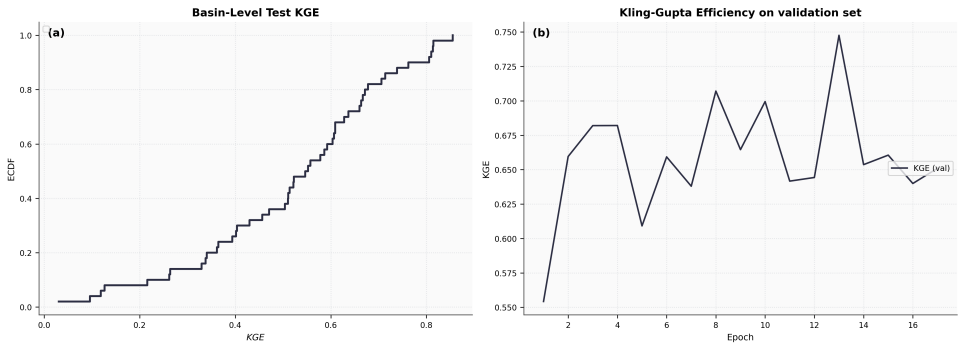

Figure 8: (a) Empirical cumulative distribution function (ECDF) of basin-level Kling–Gupta Efficiency (KGE) values across the test set, illustrating the distribution of model performance over all basins. (b) Evolution of KGE on the validation set during training, showing model convergence and stability across epochs.

2. Plot predicted vs. observed streamflow time series for at least one test basin.

   The model accurately captures the timing, magnitude, and recession behavior of streamflow events across the full period, including peak flows and baseflow conditions. Minor discrepancies are observed during some rising limbs, but overall agreement between observed and simulated hydrographs demonstrates strong model skill in reproducing basin-scale hydrological dynamics.

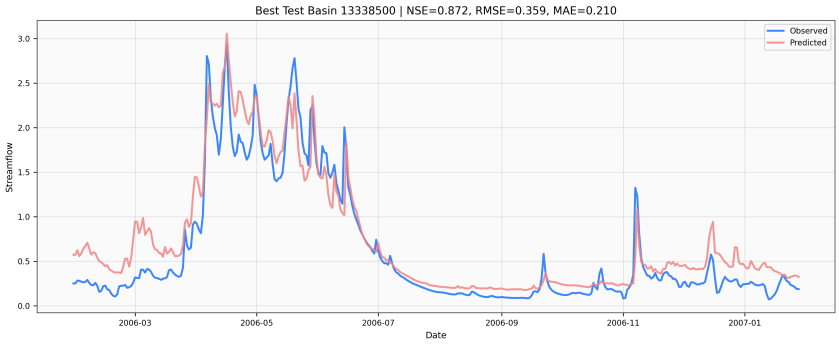

Figure 9: Observed and predicted streamflow time series for the best-performing basin (ID 13338500) in the test dataset, evaluated using the Nash–Sutcliffe Efficiency (NSE = 0.872), Root Mean Square Error (RMSE = 0.359), and Mean Absolute Error (MAE = 0.210).

The model exhibits substantial overestimation of peak flows and delayed recession dynamics, particularly during the major event in late summer. While low-flow conditions are reasonably captured, the inability to accurately reproduce high-flow magnitude and timing results in poor overall model performance. This example highlights the model's limitations in representing extreme hydrological responses in certain basins.

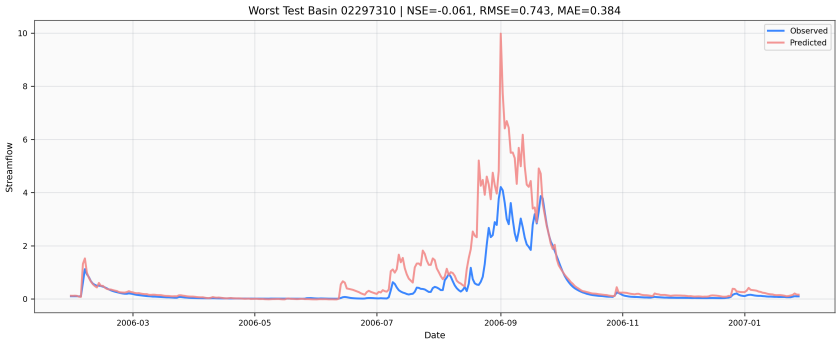

Figure 10: Observed and predicted streamflow time series for the worst-performing basin (ID 02297310) in the test dataset, evaluated using the Nash–Sutcliffe Efficiency (NSE = -0.061), Root Mean Square Error (RMSE = 0.743), and Mean Absolute Error (MAE = 0.384).

3. Create at least one parity plot or scatterplot of predicted vs. observed values.

   The clustering of points along the 1 : 1 line presented in Fig. 11 suggests generally good model performance for low to moderate flows, while increased scatter and deviation at higher flows indicate reduced accuracy in capturing extreme events. The tendency for underestimation at high observed values highlights limitations in reproducing peak streamflow magnitudes.

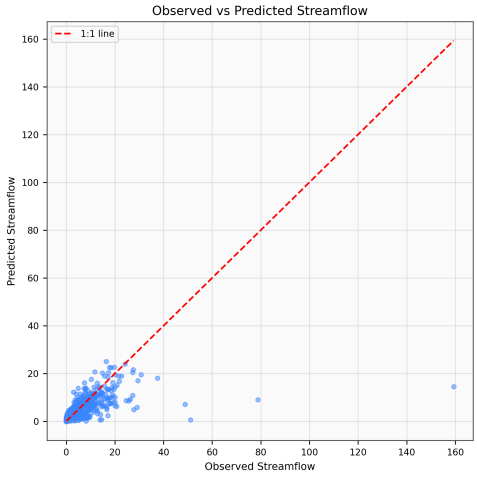

Figure 11: Observed versus predicted streamflow for all basins in the test dataset. Each point represents a paired observation–prediction value, with the red dashed line indicating the 1 : 1 relationship.

4. Identify one basin where the model performs well and one where it performs poorly.

   The results suggest a tendency for reduced model skill in more arid basins, while humid basins generally exhibit higher and more consistent performance, indicating that hydroclimatic conditions play a significant role in model predictability.

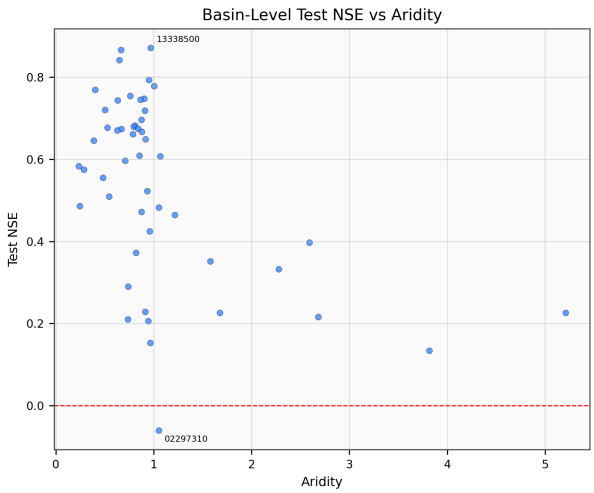

Figure 12: Relationship between basin-level model performance, measured by the Nash–Sutcliffe Efficiency (NSE), and climatic aridity across the MiniCAMELS dataset. Each point represents an individual basin, with selected basins annotated to highlight the best (ID 13338500) and worst (ID 02297310) model performance. The horizontal dashed line indicates the threshold of NSE = 0, below which model predictions perform worse than the mean observed value.

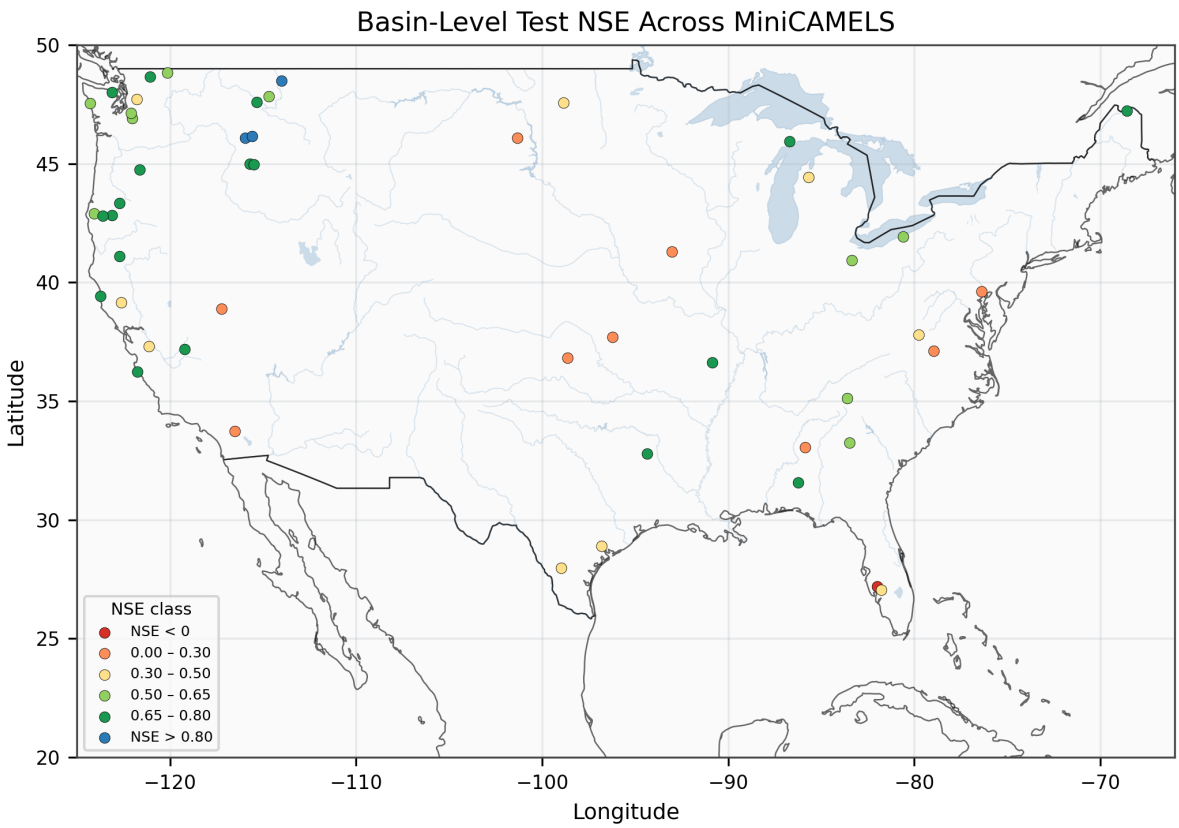

Figure 13: Spatial distribution of basin-level model performance across the MiniCAMELS dataset, evaluated using the Nash–Sutcliffe Efficiency (NSE). Each point represents an individual catchment, colored according to NSE classification ranges from poor performance (NSE ≤ 0) to excellent performance (NSE ≥ 0.80).

5. Discuss possible reasons for the performance differences, using basin behavior, climate, seasonality, or catchment attributes where relevant. Your discussion should connect the results back to both the data and the model design.

   The observed variability in model performance across basins can be attributed to differences in hydroclimatic conditions, catchment characteristics, and the inherent limitations of the model structure. Basins located in humid or snow-influenced regions tend to exhibit higher NSE and KGE values, likely due to more persistent hydrological signals and stronger seasonality, which are easier for the model to learn from the available forcing data. In contrast, arid and semi-arid basins show reduced performance, as streamflow in these systems is often dominated by intermittent events, nonlinear runoff generation processes, and higher sensitivity to localized precipitation, all of which are more difficult to capture using coarse or aggregated inputs.

   Seasonality also plays a critical role: basins with well-defined seasonal cycles (e.g., snowmelt-driven regimes) provide more predictable temporal patterns, whereas rainfall-driven basins with sporadic high-intensity events introduce greater uncer-

tainty. This is reflected in the model's tendency to underestimate or misrepresent peak flows, as seen in the parity and hydrograph analyses. Additionally, catchment attributes such as drainage area, elevation, and baseflow index influence flow persistence and variability, further contributing to differences in predictive skill.

From a modeling perspective, the sequence-based architecture is well-suited to capturing smooth temporal dynamics but may struggle with abrupt transitions and extreme events, particularly when these are underrepresented in the training data. Moreover, the reliance on lumped or basin-averaged inputs limits the model's ability to represent spatial heterogeneity within catchments.

The manner in which static attributes are incorporated may also contribute to performance limitations. In this implementation, static basin characteristics are encoded through a separate multilayer perceptron and fused with the LSTM only at the final stage of the model. While this late-fusion approach provides a compact embedding of basin properties, it does not directly influence the temporal feature extraction performed by the LSTM. As a result, the model may not fully capture how basin attributes modulate hydrological responses over time (e.g., how soil properties or aridity affect runoff generation during specific meteorological conditions).

Additionally, the strategy used for handling missing values may introduce limitations. Missing observations were removed prior to sequence construction, ensuring that all retained samples are numerically complete. However, this preprocessing step can disrupt the original temporal continuity of the data by effectively concatenating non-consecutive days, potentially altering the physical interpretation of the input sequences. As a result, the LSTM may learn temporal dependencies that do not strictly reflect true hydrological memory. Alternative approaches, such as preserving the full time axis and discarding only incomplete sequence windows, could better maintain temporal consistency and improve the physical realism of the learned relationships.